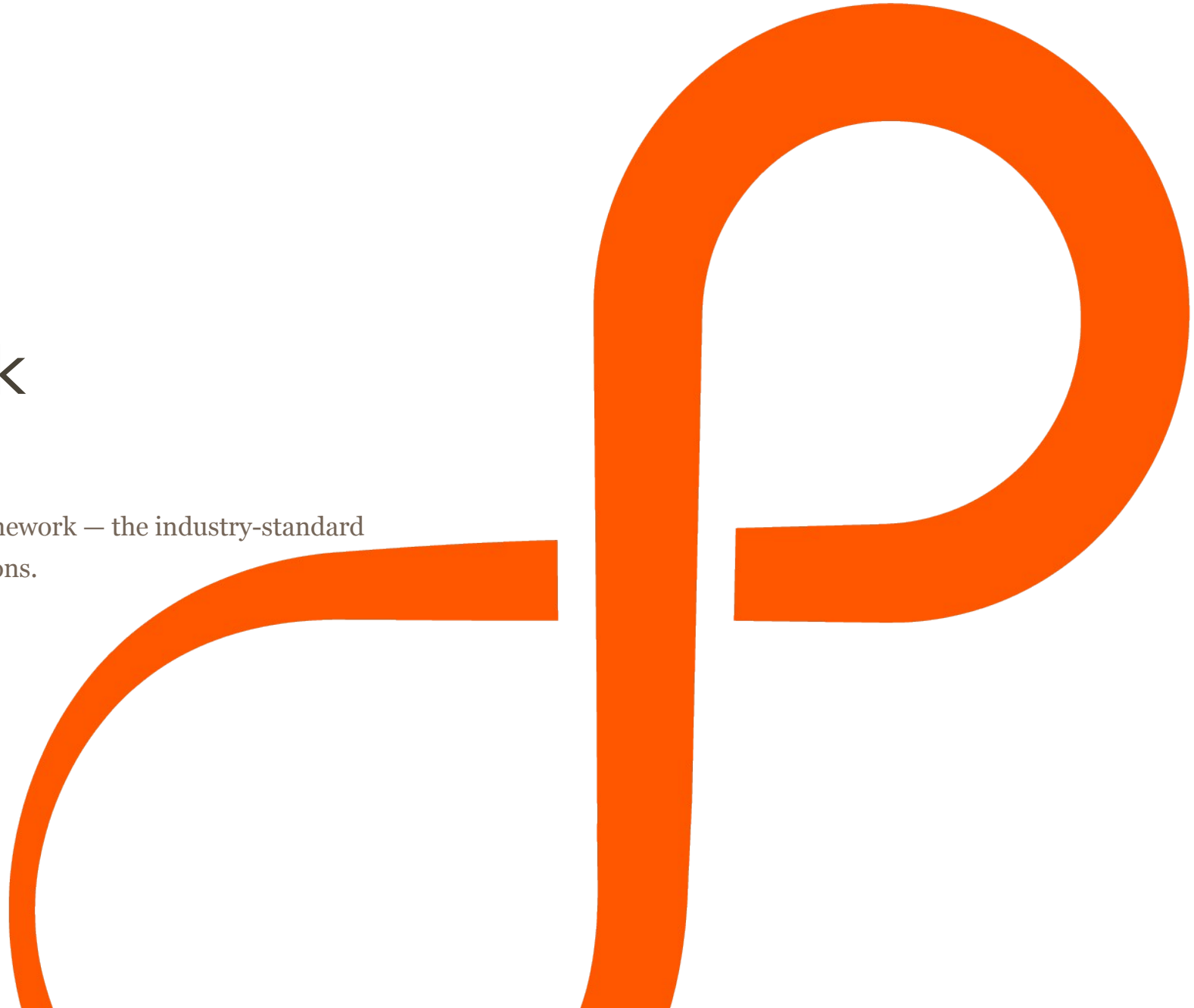
Persistent

Spring Framework

MODULE 1 | DURATION: 45 MINUTES

A comprehensive introduction to the Spring Framework — the industry-standard foundation for building enterprise Java applications.



Learning Objectives

After completing this module, students will be able to:

01

Understand Why Spring Was Created

Recognize the real-world problems that motivated Spring's development.

02

Identify Traditional Java Challenges

Explain tight coupling, manual object creation, and dependency management issues.

03

Explain Spring Architecture & Modules

Describe the layered architecture and key modules Spring provides.

04

Understand IoC & Dependency Injection

Explain Inversion of Control and Dependency Injection at a high level.

Why Do We Need Spring?

Enterprise Java applications are inherently complex. Without a framework, developers spend most of their time writing **infrastructure code** instead of business logic.

- Database Connectivity
- Security & Transactions
- Logging & Configuration
- REST APIs & Testing

Problems in Traditional Java Applications

Tight Coupling

Classes directly create dependent objects. Changing an implementation requires modifying multiple files across the codebase.

Difficult Testing

Dependencies cannot be easily replaced with mock objects, making unit testing painful and unreliable.

Manual Object Creation

Every object is created using `new`. Large applications manually instantiate thousands of objects.

Bloated Main Methods

Initialization code grows enormous. Business logic becomes buried inside setup boilerplate.

The Bloated Main Method Problem

In traditional Java, all wiring happens manually in `main()`, hiding business logic inside setup code:

```
public static void main(String[] args) {
    Database db         = new Database();
    StudentRepository r = new StudentRepository(db);
    StudentService s    = new StudentService(r);
    StudentController c = new StudentController(s);
    c.start();
}
```

⚠ As applications grow, this pattern becomes unmanageable — every new dependency must be wired by hand.

Traditional Java Architecture

The Dependency Chain

→ StudentController

→ StudentService

→ new StudentRepository()

→ new JDBC Connection()

→ Database

Characteristics

- Every class creates the next object
- Strong, rigid dependency chain
- Hard to modify or extend
- Difficult to maintain at scale

A single change — like swapping MySQL for MongoDB — forces modifications across the Repository, Service, test code, and configuration.

The Tight Coupling Problem

Consider replacing **MySQL** with **MongoDB**. Without Spring, a single infrastructure decision cascades into widespread code changes:

Repository

MySQL-specific query code must be rewritten

Service Layer

References to repository implementation must change

Test Code

All mocks and stubs must be updated

Configuration

Connection strings and drivers must be reconfigured

A small infrastructure change impacts every layer of the application.

Example Without Spring

```
public class StudentService {
    private StudentRepository repository =
        new StudentRepository(); // tightly coupled!

    public void addStudent(Student s) {
        repository.save(s);
    }
}
```

Service creates Repository directly

The implementation is hardcoded — no flexibility.

Repository cannot be replaced

Swapping implementations requires source code changes.

Mocking is difficult

Unit testing becomes unreliable and complex.

What is Spring Framework?

Spring is a **lightweight, open-source Java framework** for building enterprise applications.

- Dependency Injection
- Inversion of Control
- Transaction Management
- Database Support
- Web & Security
- Microservice Support

Key Features of Spring

Lightweight & Modular

Only required modules are used. Different modules work independently without bloating the application.

Dependency Injection

Dependencies are automatically injected, enabling loose coupling and easier testing.

IoC Container

Creates and manages Java objects automatically, removing the burden of manual instantiation.

POJO Based

Works with Plain Old Java Objects — no special base classes or interfaces required.

What is Inversion of Control (IoC)?

Traditional Java

The **application** creates objects:

```
StudentService s =
  new StudentService();
StudentRepository r =
  new StudentRepository();
```

Developer is responsible for every object's lifecycle.

With Spring (IoC)

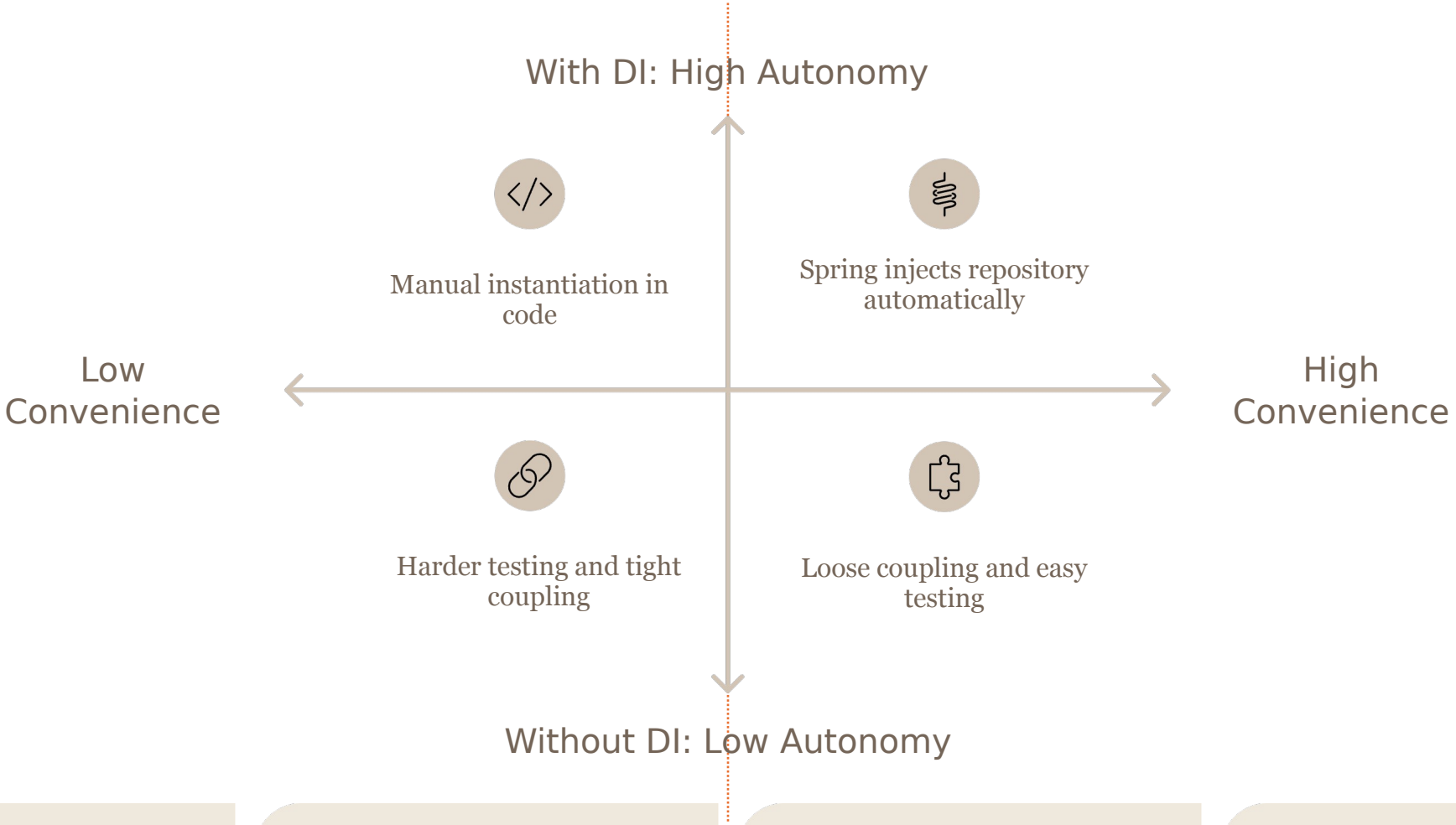
The **Spring Container** creates objects:

```
// Spring handles this:
ApplicationContext ctx =
  new ClassPathXmlApplicationContext
  ("beans.xml");
```

Instead of creating objects yourself, you ask Spring — and Spring manages them.

Dependency Injection — Introduction

Dependency Injection means Spring **automatically provides** required objects instead of the developer creating them manually.

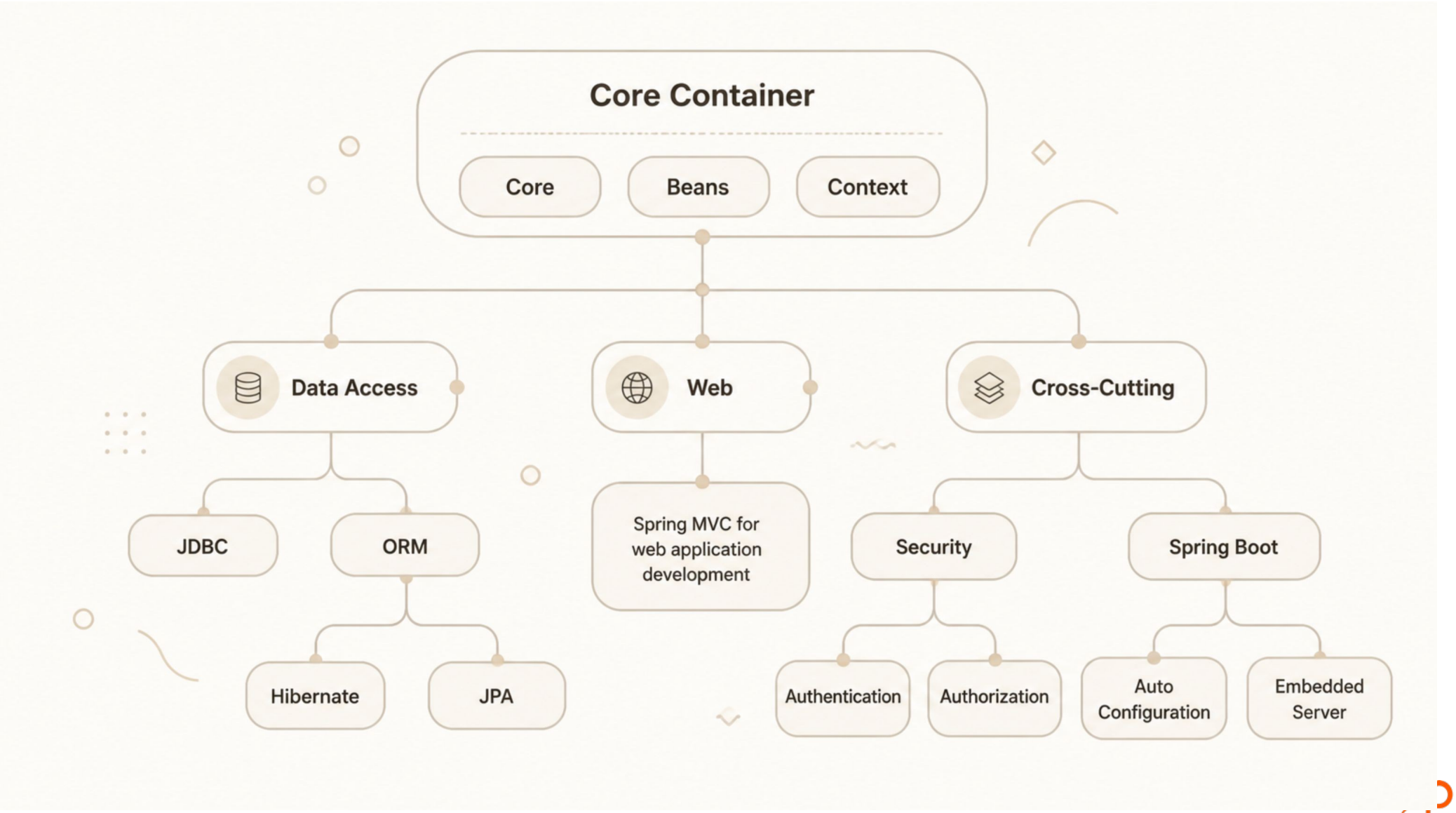


Loose Coupling | Easier Testing | Better Maintenance | Reusable Components

Persistent

Spring Framework Modules

Spring is organized into focused modules — use only what your application needs.



Spring Core Modules — Deep Dive

Core & Beans

Fundamental utilities. Creates and manages Spring beans — the backbone of every Spring application.

JDBC & ORM

Simplifies database programming. Supports Hibernate and JPA for object-relational mapping.

Spring MVC

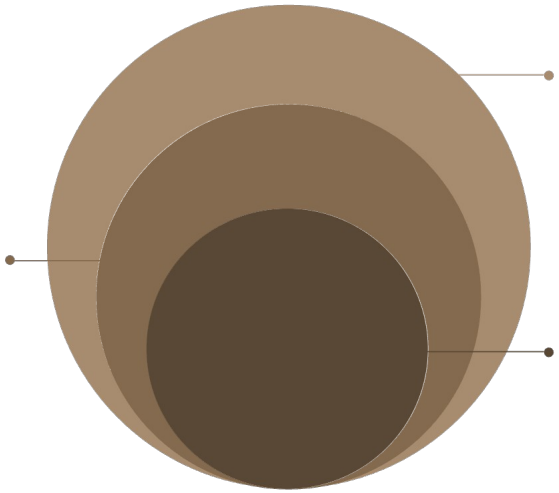
Full-featured web application development framework following the Model-View-Controller pattern.

Security & Spring Boot

Authentication, authorization, and JWT integration. Spring Boot adds auto-configuration and embedded server for rapid development.

Persistent

High-Level Spring Architecture



Spring IoC Container (ApplicationContext)

- **Creates Beans**

  Instantiates all application objects automatically.

- **Injects Dependencies**

  Wires beans together without manual code.

- **Manages Bean Lifecycle**

  Controls initialization and destruction.

- **Provides Configuration**

  Centralizes all application configuration.

Benefits of Spring

Loose Coupling

Easier Testing

Reusable Components

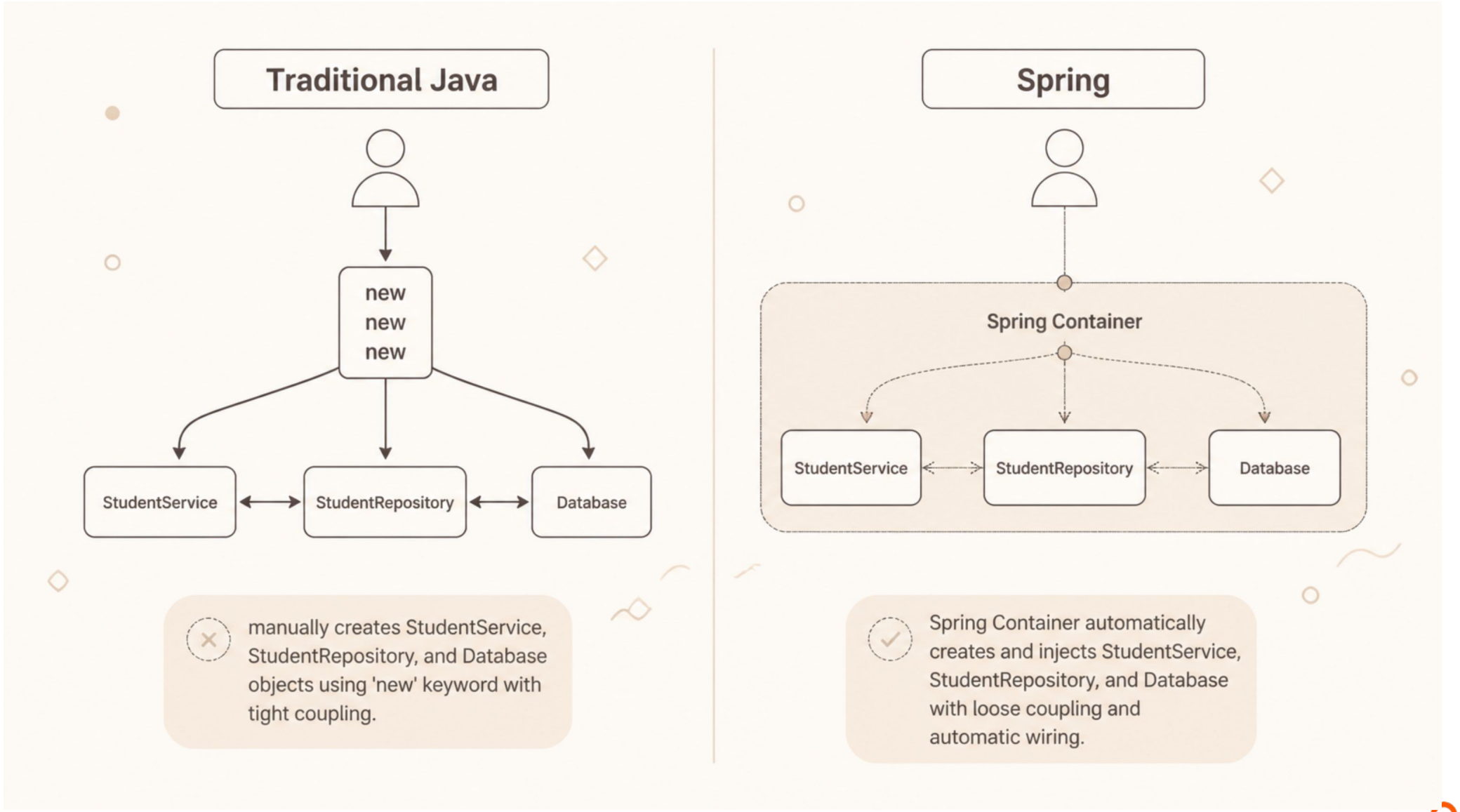
Better Maintainability

Faster Development

Enterprise Ready & Scalable

Persistent

Traditional Java vs. Spring — Side by Side



Spring shifts object creation responsibility from the developer to the framework — enabling cleaner, more maintainable code.

IoC & DI — Putting It Together

Inversion of Control

Control of object creation is **inverted** — instead of the application creating objects, the **Spring IoC Container** creates and manages them.

You define *what* you need. Spring decides *how* to provide it.

Dependency Injection

DI is the **mechanism** through which IoC is implemented. Spring injects the right dependency at the right time — automatically.

- Constructor Injection
- Setter Injection
- Field Injection (`@Autowired`)

IoC is the **principle**; Dependency Injection is the **pattern** that implements it.

Module Summary

In this module, you covered the foundational concepts that make Spring the industry standard for enterprise Java development:

1. **Why Spring Was Created**
   To eliminate infrastructure boilerplate and let developers focus on business logic.
2. **Problems with Traditional Java**
   Tight coupling, manual object creation, difficult testing, and bloated main methods.
3. **Spring Framework Overview**
   Lightweight, modular, open-source framework with a powerful IoC container.
4. **Spring Modules**
   Core, Beans, Context, JDBC, ORM, MVC, Security, and Spring Boot.
5. **IoC & Dependency Injection**
   Spring manages object creation and automatically injects dependencies.

What's Next?

MODULE 2 PREVIEW

Spring IoC Container & Dependency Injection

In the next module, you will get **hands-on experience** with Beans and ApplicationContext — configuring the IoC container, defining beans, and wiring dependencies using both XML and annotation-based approaches.

Bean Configuration

XML & Annotation-based

ApplicationContext

Loading and using the container

Hands-on Lab

Build your first Spring application